Jr.Pen 5.2. Content Discovery

Task 1 - What Is Content Discovery?

Content can be many things, a file, video, picture, backup, a website feature. When we talk about content discovery, we're not talking about the obvious things we can see on a website; it's the things that aren't immediately presented to us and that weren't always intended for public access. This content could be, for example, pages or portals intended for staff usage, older versions of the website, backup files, configuration files, administration panels, etc. There are three main ways of discovering content on a website which we'll cover. Manually, Automated and OSINT

Task 2 - Manual Discovery - Robots.txt

The robots.txt file is a document that tells search engines which pages they are and aren't allowed to show on their search engine results or ban specific search engines from crawling the website altogether. It can be common practice to restrict certain website areas so they aren't displayed in search engine results. These pages may be areas such as administration portals or files meant for the website's customers. This file gives us a great list of locations on the website that the owners don't want us to discover as penetration testers.

Task 3 - Manual Discovery - Favicon

The favicon is a small icon displayed in the browser's address bar or tab used for branding a website. Sometimes when frameworks are used to build a website, a favicon that is part of the installation gets leftover, and if the website developer doesn't replace this with a custom one, this can give us a clue on what framework is in use.

```
user@machine$ curl https://static-labs.tryhackme.cloud/sites/
favicon/images/favicon.ico | md5sum
```

cURL tool to send an HTTP request to tryhackme's homepage and display the raw HTML source code in the terminal. calculate the MD5 checksum of the HTML content currently served by tryhackme.

Task 4 - Manual Discovery - Sitemap.xml

the sitemap.xml file gives a list of every file the website owner wishes to be listed on a search engine. These can sometimes contain areas of the website that are a bit more difficult to navigate to or even list some old webpages that the current site no longer uses but are still working behind the scenes.

Task 5 - Manual Discovery - HTTP Headers

HTTP Headers: When we make requests to the web server, the server returns various HTTP headers. These headers can sometimes contain useful information such as the webserver software and possibly the programming/scripting language in use. In the below example, we can see the webserver is NGINX version 1.18.0 and runs PHP version 7.4.3. Using this information, we could find vulnerable versions of software being used.

Task 6 - Manual Discovery - Framework Stack

Framework Stack: Once you've established the framework of a website, either from the above favicon example or by looking for clues in the page source such as comments, copyright notices or credits, you can then locate the framework's website. From there, we can learn more about the software and other information, possibly leading to more content we can discover.

Task 7 - OSINT - Google Hacking / Dorking

Google hacking / Dorking utilizes Google's advanced search engine features, which allow you to pick out custom content. You can, for instance, pick out results from a certain domain name using the **site:** filter, for example (site:tryhackme.com) you can then match this up with certain search terms, say, for example, the word admin (site:tryhackme.com admin) this then would only return results from the tryhackme.com website which contain the word admin in its content.

Filter	Example	Description
site	site:tryhackme.com	returns results only from the specified website address
inurl	inurl:admin	returns results that have the specified word in the URL

filetype	filetype:pdf	returns results which are a particular file extension
intitle	intitle:admin	returns results that contain the specified word in the title

Task 8 - OSINT - Wappalyzer

Wappalyzer (https://www.wappalyzer.com/(opens in new tab)) is an online tool and browser extension that helps identify what technologies a website uses, such as frameworks, Content Management Systems (CMS), payment processors and much more, and it can even find version numbers as well.

Task 9 -OSINT - Wayback Machine

The Wayback Machine (https://archive.org/web/(opens in new tab)) is a historical archive of websites that dates back to the late 90s. You can search a domain name, and it will show you all the times the service scraped the web page and saved the contents. This service can help uncover old pages that may still be active on the current website.

Task 10 - OSINT - GitHub

GitHub: To understand GitHub, you first need to understand Git. Git is a **version control system** that tracks changes to files in a project. Working in a team is easier because you can see what each team member is editing and what changes they made to files. When users have finished making their changes, they commit them with a message and then push them back to a central location (repository) for the other users to then pull those changes to their local machines. GitHub is a hosted version of Git on the internet. Repositories can either be set to public or private and have various access controls

Task 11 - OSINT - S3 Buckets

S3 Buckets are a storage service provided by Amazon AWS, allowing people to save files and even static website content in the cloud accessible over HTTP and HTTPS. The owner of the files can set access permissions to either make files public, private and even writable. Sometimes these access permissions are incorrectly set and inadvertently allow access to files that shouldn't be available to the public. The format of the S3 buckets
is http(s)://**{name}.s3.amazonaws.com**(opens in new tab) where {name} is

decided by the owner, such as tryhackme-assets.s3.amazonaws.com(opens in new tab). S3 buckets can be discovered in many ways, such as finding the URLs in the website's page source, GitHub repositories, or even automating the process.

Task 12 - Automated Discovery

What is Automated Discovery?

Automated discovery is the process of using tools to discover content rather than doing it manually. This process is automated as it usually contains hundreds, thousands or even millions of requests to a web server. These requests check whether a file or directory exists on a website, giving us access to resources we didn't previously know existed. This process is made possible by using a resource called wordlists.

What are wordlists?

Wordlists are just text files that contain a long list of commonly used words; they can cover many different use cases. For example, a password wordlist would include the most frequently used passwords, whereas we're looking for content in our case, so we'd require a list containing the most commonly used directory and file names. An excellent resource for wordlists that is preinstalled on the THM AttackBox is https://github.com/danielmiessler/SecLists(opens in new tab) which Daniel Miessler curates.

Automation Tools

Although there are many different content discovery tools available, all with their features and flaws, we're going to cover three which are preinstalled on our attack box, ffuf, dirb and gobuster.

```
user@machine$ ffuf -w /usr/share/wordlists/SecLists/Discover
y/Web-Content/common.txt -u http://MACHINE_IP/FUZZ
user@machine$ dirb http://MACHINE_IP/ /usr/share/wordlists/Se
cLists/Discovery/Web-Content/common.txt
user@machine$ gobuster dir --url http://MACHINE_IP/ -w /usr/s
hare/wordlists/SecLists/Discovery/Web-Content/common.txt
```

ffuf: used to discover vulnerabilities, hidden directories, parameters, and misconfigurations in web applications, APIs, and networks. It works by sending a large number of automated requests with different inputs to analyze how the application responds. **GET/POST parameter fuzzing** on Linux systems, uses the **SecLists** wordlist to test common filenames

-w wordlist file used for fuzzing, -u target URL

Dirb: is a command-line web content scanner used to discover existing and hidden files and directories on a web server by launching dictionary-based brute-force attacks. It operates by sending HTTP requests to a target URL using words from a specified **wordlist** and analyzing the HTTP response codes to identify accessible resources.

gobuster dir: command is the **directory/file enumeration mode** of the Gobuster tool, designed to discover hidden directories and files on a web server by brute-forcing URIs using a specified wordlist.